

FinTrack

Project	FinTrack — a single-user web application for tracking personal income and expenses.
Document	Business Requirements Document (BRD) — v1.0
Audience	Frontend developer (implementation), Backend developer (API contract)
Status	Approved for development
Related docs	API Contract (to follow), Frontend Technical Requirements (to follow), UI Design (to follow)

Purpose of this document. This BRD describes *what* FinTrack must do from a business and user perspective — not how it is built. It is the single source of truth from which the API contract, the frontend architecture, and the UI design are derived. Each requirement is written as a user story with acceptance criteria so that progress can be verified objectively.

1. Product Overview

FinTrack lets a person record their money movements — income and expenses — organise them into categories and accounts, and understand their financial situation through filters and visual analytics. It is a personal tool: each registered user sees only their own data. There is no admin role, no sharing, and no multi-user collaboration.

Primary goal for the user: "Where does my money go, and how much do I have?"

Scope boundaries. The following are explicitly *out of scope* for v1: multi-currency support, bank integrations / automatic imports, shared or family budgets, mobile native apps, admin panel, and recurring-transaction automation (may be considered later).

2. User Roles

There is a single role in the system:

- **Registered User** — can register, log in, and perform all operations on their own data (transactions, categories, accounts, budgets). A user can never see or affect another user's data.

Unauthenticated visitors can only access the public pages: the landing/login and registration screens.

3. Domain Model (business entities)

These are the core concepts the product deals with. Exact field types and API shapes live in the API Contract; this is the business-level description.

Entity	Description	Key attributes (business view)
User	The account owner.	Email, password, display name, creation date.
Account	A place where money is held (e.g. cash, debit card, savings). Every transaction belongs to exactly one account.	Name, type (cash / card / savings / other), starting balance, owner.
Category	A label that classifies a transaction (e.g. Groceries, Salary, Rent). Each category is either an income category or an expense category.	Name, type (income / expense), colour, icon (optional), owner.
Transaction	A single money movement — the central entity of the product.	Type (income / expense), amount, date, category, account, note (optional), owner.
Budget	A monthly spending limit set for a specific expense category.	Category, limit amount, month, owner.

Could-have

Business rule: a transaction's category type must match its transaction type (an expense transaction can only use an expense category, and vice versa).

4. Functional Requirements

Requirements are grouped into epics. Each story carries a priority label: **MUST** (required for release), **SHOULD** (important but not blocking), **COULD** (optional / stretch).

Epic A — Authentication & Account

Users must be able to create an account, sign in securely, stay signed in across reloads, and sign out.

US-A1 Register a new account

MUST

As a new visitor, **I want** to register with my email and a password, **so that** I can start tracking my finances.

Acceptance criteria:

- The registration form requires email, password, and display name.
- Email must be valid in format and not already registered; a clear error is shown otherwise.
- Password must meet a minimum strength rule (e.g. at least 8 characters).
- On success, the user is automatically logged in and taken to the dashboard.

US-A2 Log in

MUST

As a registered user, **I want** to log in with my email and password, **so that** I can access my data.

Acceptance criteria:

- Correct credentials grant access and redirect to the dashboard.
- Incorrect credentials show a generic "invalid email or password" message (no hint which field is wrong).
- The login state persists across page reloads without asking the user to log in again.

US-A3 Stay logged in / session restore

MUST

As a logged-in user, **I want** my session to be restored silently when I reopen or refresh the app, **so that** I am not forced to log in repeatedly.

Acceptance criteria:

- When a short-lived session expires, it is renewed automatically in the background without interrupting the user.
- If the session can no longer be renewed, the user is redirected to the login screen.

US-A4 Log out

MUST

As a logged-in user, **I want** to log out, **so that** my data is protected on a shared device.

Acceptance criteria:

- Logging out ends the session and redirects to the login screen.
- After logout, protected pages are no longer accessible without logging in again.

US-A5 Protected areas

MUST

As the product owner, **I want** all financial data pages to require authentication, **so that** data is never exposed to anonymous visitors.

Acceptance criteria:

- Any attempt to open a protected page while logged out redirects to login.
- After a successful login the user is returned to the page they originally requested.

Epic B — Accounts (wallets)

Users manage the places their money lives.

US-B1 Manage accounts (CRUD)

MUST

As a user, **I want** to create, view, edit, and delete my accounts, **so that** I can mirror my real wallets and cards.

Acceptance criteria:

- I can create an account with a name, type, and starting balance.
- I can see a list of all my accounts with their current balance.
- I can edit an account's name, type, and starting balance.
- Deleting an account is blocked (or requires confirmation) if it still has transactions, to avoid orphaned data.

US-B2 See per-account balance

SHOULD

As a user, **I want** each account to show its current balance (starting balance +/- its transactions), **so that** I know how much is in each wallet.

Acceptance criteria:

- Balance updates automatically when transactions on that account are added, edited, or removed.

Epic C — Categories

Users classify their money movements.

US-C1

Manage categories (CRUD)

MUST

As a user, **I want** to create, view, edit, and delete income and expense categories, **so that** I can organise my transactions meaningfully.

Acceptance criteria:

- Each category has a name, a type (income or expense), and a colour.
- Categories are listed grouped by type.
- A category in use by transactions cannot be silently deleted; the user is warned and must confirm or reassign.

US-C2

Sensible defaults on first login

SHOULD

As a new user, **I want** a small set of default categories to already exist, **so that** I can start recording transactions immediately without setup.

Acceptance criteria:

- On account creation, a few common categories (e.g. Salary, Groceries, Rent, Transport) are pre-populated.
- The user can edit or delete these like any other category.

Epic D — Transactions (core)

The heart of the product: recording and reviewing money movements.

US-D1

Add a transaction

MUST

As a user, **I want** to record an income or expense with amount, date, category, account, and an optional note, **so that** my history stays accurate.

Acceptance criteria:

- Amount must be a positive number; the form prevents zero/negative and non-numeric input.
- The category list is filtered to match the chosen type (expense form shows only expense categories).
- Date defaults to today but can be changed; future dates are allowed but flagged visually.
- On save, the transaction appears immediately in the list and the relevant balances/totals update.

US-D2

View transaction list

MUST

As a user, **I want** to see my transactions in a paginated, sorted list, **so that** I can review my history without the page becoming slow.

Acceptance criteria:

- Transactions are shown newest-first by default, each with type, amount, category, account, date, and note.
- The list is paginated (or infinite-scrolled); it does not load the entire history at once.
- Income and expense are visually distinguishable (e.g. colour / sign).

US-D3

Edit and delete a transaction

MUST

As a user, **I want** to edit or delete a transaction, **so that** I can correct mistakes.

Acceptance criteria:

- Editing pre-fills the form with existing values.
- Deletion requires a confirmation step.
- Any change is reflected immediately in the list, balances, and analytics.

US-D4

Filter and search transactions

MUST

As a user, **I want** to filter my transactions by date range, type, category, and account, **so that** I can find and analyse specific spending.

Acceptance criteria:

- Filters can be combined (e.g. "expenses in Groceries on the card, last month").
- The active filters are visible and can be cleared with one action.
- Filtering updates the list and the visible totals for the filtered set.

Epic E — Dashboard & Analytics

Turning raw transactions into understanding.

US-E1 Summary dashboard

MUST

As a user, **I want** a dashboard showing my total balance, total income, and total expenses for a chosen period, **so that** I get an at-a-glance picture.

Acceptance criteria:

- The dashboard shows: overall balance across accounts, total income and total expense for the selected period, and the resulting net.
- A period selector allows: this month, last month, and a custom date range.
- Figures recalculate when the period changes.

US-E2 Spending by category (chart)

MUST

As a user, **I want** to see how my expenses break down by category as a chart, **so that** I can spot where money goes.

Acceptance criteria:

- A pie/donut chart shows expense share per category for the selected period.
- Categories use their assigned colours.
- An empty state is shown when there is no data for the period.

US-E3 Income vs expense over time (chart)

SHOULD

As a user, **I want** to see income and expense trends across months as a bar or line chart, **so that** I can understand my direction over time.

Acceptance criteria:

- A chart plots monthly income and expense side by side for the last several months.
- Hovering a data point reveals the exact figures.

Epic F — Budgets Could-have / stretch

Optional advanced module. Include only after Epics A-E are complete.

US-F1 Set a monthly budget per category

COULD

As a user, **I want** to set a monthly spending limit for an expense category, **so that** I can control overspending.

Acceptance criteria:

- I can assign a limit amount to an expense category for a given month.
- A progress indicator shows how much of the budget is used.
- When spending exceeds the limit, the indicator changes to a warning state.

5. Non-Functional Requirements

Area	Requirement
Security	All financial data is behind authentication. A user can only ever read or modify their own records. Passwords are never stored in plain text (backend concern).
Usability	Every destructive action (delete) requires confirmation. Forms show inline validation errors. Loading and empty states are handled explicitly, never left blank.
Feedback	Every user action produces visible feedback within ~1 second (loading indicator, success, or error). Network/save failures are communicated clearly, not swallowed.
Data integrity	Amounts are handled as exact values (currency), never as approximate floating figures that drift. Category/type consistency rules are enforced.
Performance	Transaction lists are paginated so the app stays responsive regardless of history size.
Responsiveness	The layout is usable on a modern laptop screen at minimum; graceful narrowing is a plus but not required for v1.
Localisation	Single currency for v1. Dates and amounts are formatted consistently for display.

6. Release Scope Summary

The v1 deliverable is considered **done** when all MUST and SHOULD stories across Epics A-E pass their acceptance criteria. Epic F (Budgets) is an optional stretch to be picked up only afterwards.

Epic	In v1?	Notes
A — Auth & Account	Yes (core)	Real JWT-based auth with silent session renewal.
B — Accounts	Yes (core)	Full CRUD + balance.
C — Categories	Yes (core)	Full CRUD + defaults.
D — Transactions	Yes (core)	CRUD + filter/search + pagination. The centrepiece.
E — Dashboard & Analytics	Yes (core)	Summary + at least the category breakdown chart.
F — Budgets	Stretch	Only after A-E are solid.

End of Business Requirements Document v1.0. The next document in the set is the API Contract, which translates these entities and stories into concrete endpoints, request/response shapes, and error codes.